

Technical Documentation

Enterprise Microservices Backend — Architecture, APIs, Security & Learning Guide

Zomato / Swiggy-style system built with Java 17 and Spring Boot 3

Project: food-delivery-platform (Maven Multi-Module)

Version: 1.0.0-SNAPSHOT

Modules: 12 (11 microservices + common-lib)

Stack: Spring Cloud, Eureka, JWT, MySQL, OpenFeign

Table of Contents

1. Executive Summary
2. Technology Stack
3. System Architecture
4. Microservices Overview
5. Layered Architecture
6. Project Structure
7. Security and JWT Authentication
8. Domain Entities and DTOs
9. Inter-Service Communication (Feign)
10. Business Flows
11. REST API Reference
12. Exception Handling
13. Setup and Deployment
14. Learning Guide

1. Executive Summary

This project is an enterprise-grade Food Delivery Microservices Backend similar in concept to Zomato or Swiggy. It is organized as a Maven multi-module monorepo with eleven deployable Spring Boot applications plus a shared common library.

Key design principles:

- Microservices architecture — each service owns its database.
- Layered architecture — controller, service, repository in every service.
- DTO-only APIs — JPA entities are never exposed over HTTP or Feign.
- Eureka service discovery and Spring Cloud Config for centralized settings.
- API Gateway as a single entry point with load-balanced routing.
- JWT authentication with role-based access control (RBAC).

2. Technology Stack

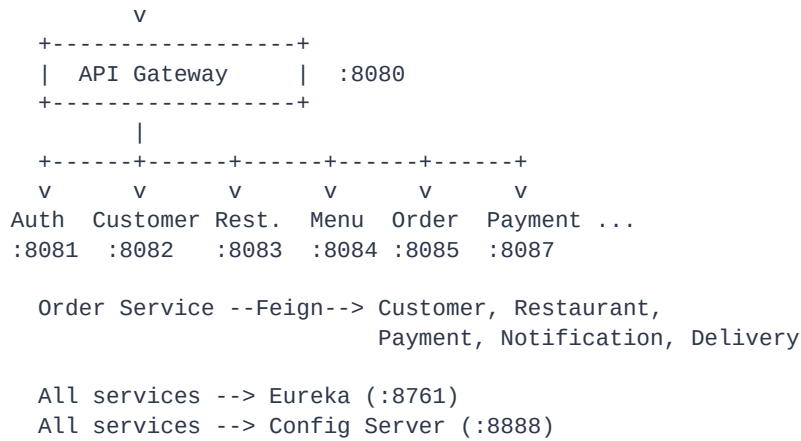
Category	Technology	Version
Language	Java	17
Framework	Spring Boot	3.3.5
Cloud	Spring Cloud	2023.0.3
Database	MySQL + Spring Data JPA	Per service
Security	Spring Security + JWT	jjwt 0.12.6
Inter-service	OpenFeign	order-service
Discovery	Netflix Eureka	Port 8761
Config	Spring Cloud Config	Port 8888
Gateway	Spring Cloud Gateway	Port 8080
API Docs	SpringDoc OpenAPI	Swagger UI
Build	Maven	Multi-module

3. System Architecture

Clients connect only to the API Gateway (port 8080). The gateway uses Eureka to find service instances and routes requests. The Order Service orchestrates other services using OpenFeign when a customer places an order.

3.1 Architecture Diagram

[Client Apps]
|



3.2 Service Ports and Databases

Service	Port	Database	Role
discovery-server	8761	-	Eureka registry
config-server	8888	-	Central configuration
api-gateway	8080	-	API routing
auth-service	8081	auth_db	JWT login/register
customer-service	8082	customer_db	Customer CRUD
restaurant-service	8083	restaurant_db	Restaurants
menu-service	8084	menu_db	Menu items
order-service	8085	order_db	Order orchestration
delivery-partner-service	8086	delivery_db	Delivery
payment-service	8087	payment_db	Payments
notification-service	8088	notification_db	Alerts

4. Microservices Overview

4.1 Infrastructure Services

Discovery Server: All services register on startup. Dashboard at <http://localhost:8761>

Config Server: Reads YAML from config-repo/. Each service imports config via `spring.config.import`.

API Gateway: Routes `/api/customers/**` to `customer-service`, etc. Uses `StripPrefix=1` so `/api/customers/1` becomes `/customers/1` downstream.

4.2 Common Library (common-lib)

- Enums: `Role`, `OrderStatus`, `DeliveryStatus`, `PaymentStatus`
- Exceptions: `ResourceNotFoundException`, `InvalidRequestException`, `UnauthorizedException`
- `GlobalExceptionHandler` with uniform `ErrorResponseDto`

5. Layered Architecture (Every Service)

```
controller      -> REST endpoints, ResponseEntity, @Valid
  |
  service        -> Interface (business contract)
  |
  serviceImpl    -> Business logic implementation
  |
  repository     -> Spring Data JPA
  |
  entity         -> @Entity (NEVER exposed in API)
```

Also: `dto/`, `mapper/`, `config/`, `security/`, `exception/`

Rule: Controllers use `RequestDto` and `ResponseDto` only. Mappers convert between `Entity` and `DTO` inside the service layer.

6. Project Structure

```
food-delivery-platform/
|-- pom.xml                (parent)
|-- common-lib/            (shared code)
|-- config-repo/           (YAML configs)
|-- discovery-server/
|-- config-server/
|-- api-gateway/
|-- auth-service/
|-- customer-service/
|-- restaurant-service/
|-- menu-service/
|-- order-service/         (Feign clients in client/)
|-- delivery-partner-service/
|-- payment-service/
|-- notification-service/
|-- docker-compose.yml
```

7. Security and JWT Authentication

Role	Purpose
ADMIN	Platform administration
CUSTOMER	Place orders (referenceId = customerId)
RESTAURANT_OWNER	Manage restaurants and menus
DELIVERY_PARTNER	Update delivery status

7.1 Login Flow

- POST /auth/register with email, password, role, referenceId
- Password stored with BCryptPasswordEncoder
- POST /auth/login returns JWT token
- Send header: Authorization: Bearer
- JwtAuthenticationFilter validates token on each request

7.2 Sample Auth Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "tokenType": "Bearer",
  "userId": 1,
  "email": "john@mail.com",
  "role": "CUSTOMER",
  "referenceId": 1
}
```

8. Domain Entities and DTOs

Entity	Service	Main Fields
User	auth	email, password, role, referenceId
Customer	customer	name, email, phone, address
Restaurant	restaurant	restaurantName, ownerName, address, rating
MenuItem	menu	restaurantId, itemName, price, availability
Order	order	customerId, restaurantId, totalAmount, orderStatus
DeliveryPartner	delivery	name, phone, vehicleNumber, availabilityStatus
Payment	payment	orderId, amount, paymentStatus, paymentMethod
Notification	notification	userId, message, type, sentAt

8.1 Status Enums

Enum	Values
OrderStatus	PLACED, CONFIRMED, PREPARING, READY, OUT_FOR_DELIVERY, DELIVERED, CANCELLED
DeliveryStatus	ASSIGNED, PICKED_UP, OUT_FOR_DELIVERY, DELIVERED
PaymentStatus	PENDING, SUCCESS, FAILED, REFUNDED

9. Inter-Service Communication (OpenFeign)

Only order-service calls other services. All calls use DTOs in package `com.fooddelivery.order.client` — never entities.

Feign Client	Target	Endpoint
CustomerClient	customer-service	GET /customers/{id}
RestaurantClient	restaurant-service	GET /restaurants/{id}
PaymentClient	payment-service	POST /payments/process
NotificationClient	notification-service	POST /notifications
DeliveryPartnerClient	delivery-partner-service	POST /delivery-partners/assign

10. Business Flows

10.1 Customer Registration and Login

Step 1: POST /api/customers -> Create customer (get id)

Step 2: POST /api/auth/register -> role=CUSTOMER, referenceId=<id>
Step 3: POST /api/auth/login -> Get JWT token
Step 4: Use Bearer token on API calls

10.2 Restaurant and Menu

Step 1: Register RESTAURANT_OWNER in auth-service
Step 2: POST /api/restaurants -> Create restaurant
Step 3: POST /api/menus -> Add menu items
Step 4: PUT /api/menus/{id} -> Update availability

10.3 Order Placement (Orchestration)

POST /api/orders

1. Feign: validate customer exists
2. Feign: validate restaurant exists
3. Save order (status = PLACED)
4. Feign: process payment -> SUCCESS
5. Update order (status = CONFIRMED)
6. Feign: send ORDER_PLACED notification
7. Feign: send ORDER_ACCEPTED notification

10.4 Delivery Flow

POST /api/orders/{id}/assign-delivery
-> Assigns nearest available partner (status = ASSIGNED)
-> Order status = OUT_FOR_DELIVERY

PATCH /api/delivery-partners/{id}/status
-> ASSIGNED -> PICKED_UP -> OUT_FOR_DELIVERY -> DELIVERED

11. REST API Reference

Base URL (via Gateway): **http://localhost:8080/api**

11.1 Auth Service

Method	Endpoint	Description
POST	/api/auth/register	Register with role
POST	/api/auth/login	Login, get JWT

11.2 Customer Service

Method	Endpoint	Description
POST	/api/customers	Create customer
GET	/api/customers/{id}	Get by ID
GET	/api/customers	List all
PUT	/api/customers/{id}	Update
DELETE	/api/customers/{id}	Delete

11.3 Restaurant Service

Method	Endpoint	Description
POST	/api/restaurants	Create
GET	/api/restaurants/{id}	Get by ID
GET	/api/restaurants	List all
PUT	/api/restaurants/{id}	Update
DELETE	/api/restaurants/{id}	Delete

11.4 Menu Service

Method	Endpoint	Description
POST	/api/menus	Create menu item
GET	/api/menus/{id}	Get item
GET	/api/menus/restaurant/{id}	Menu by restaurant
PUT	/api/menus/{id}	Update
DELETE	/api/menus/{id}	Delete

11.5 Order Service

Method	Endpoint	Description
POST	/api/orders	Place order
GET	/api/orders/{id}	Get order
GET	/api/orders/customer/{id}	By customer
PATCH	/api/orders/{id}/status	Update status
POST	/api/orders/{id}/assign-delivery	Assign partner

11.6 Other Services

Method	Endpoint	Service
POST	/api/delivery-partners/assign	Assign partner
PATCH	/api/delivery-partners/{id}/status	Delivery status
POST	/api/payments/process	Process payment
POST	/api/notifications	Send notification

11.7 Sample Order Request

POST http://localhost:8080/api/orders
Content-Type: application/json

```
{  
  "customerId": 1,  
  "restaurantId": 1,  
  "totalAmount": 499.0,  
  "paymentMethod": "UPI"  
}
```

12. Exception Handling

Exception	HTTP Status
ResourceNotFoundException	404 Not Found
InvalidRequestException	400 Bad Request
UnauthorizedException	401 Unauthorized
Validation errors	400 with field map

13. Setup and Deployment

13.1 Prerequisites

- Java 17+, Maven 3.8+, MySQL 8
- Optional: docker-compose up -d for MySQL (root/root)

13.2 Startup Order

1. docker-compose up -d (MySQL)
2. discovery-server (port 8761)
3. config-server (port 8888)
4. All business services
5. api-gateway (port 8080)

13.3 Build and Run

```
./mvnw clean install -DskipTests
./mvnw -pl order-service spring-boot:run
```

13.4 Swagger UI

Each service: <http://localhost:{port}/swagger-ui.html>

14. Learning Guide

Recommended study order:

- 1. common-lib — shared exceptions and enums
- 2. customer-service — simplest CRUD layers
- 3. auth-service — JWT and Spring Security
- 4. discovery, config, gateway — infrastructure
- 5. order-service — Feign orchestration
- 6. delivery-partner-service — status workflow

Food Delivery Microservices Platform — Documentation v1.0